

第4章 Verilog HDL语言基础（续）

4.5 Verilog 语句

一个Verilog程序的基本结构：

```
module module_name (port_name);
   Declarations:                    //说明部分
   reg, wire, parameter,
   input, output, inout,
   function, task, ……
   Statements:                      //语句部分
   initial statement                // initial 语句
   always statement                 // always 语句
   module instantiation             // 模块例化语句
   gate instantiation               // 基本门例化语句
   UDP instantiation                // 用户自定义例化语句
   continuous assignment            // 连续赋值语句
endmodule
```

Verilog 语句

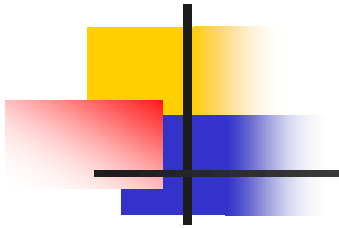
- **Verilog语句分为并行语句和顺序语句；**
- 并行语句是同时工作的，与书写顺序无关；
- 顺序语句有执行的顺序；
- 说明部分要放在使用它的语句之前；
- 语句都可以进行仿真，但只有一部分可以综合。

4.5.1 连续赋值语句

一般语法格式：

```
assign［延时］目标变量名 = 表达式;
```

- 并行语句，assign是关键字；
- 方括号内的延时可以省略；
- 目标变量必须是线网类型；
- 表达式可以是线网类型、寄存器类型或函数等；
- 赋值符号是“=”。



assign语句的执行过程是：

- 当右边表达式中任意一个变量发生了变化，就会导致表达式重新计算一次。
- 如果没有延时部分，则表达式的值立即赋给目标变量；
- 如果有延时部分，则经过给定的延时之后，表达式的值再赋给目标变量。例如：

```
assign F = D | E;
assign  #6  Ask = Q  ^ L;
```

4.5.2 过程语句

- 过程语句用于描述电路的行为，分为always语句和initial语句；
- initial语句主要用于仿真，不能进行综合；
- always语句可以综合，是使用较多的过程语句。

4.5.2.1 always语句的语法格式

```
always @(敏感信号列表)
  begin
    语句或语句块;
  end
```

always语句的执行过程是：

- 当敏感信号列表中任意一个信号有变化，就会触发always语句执行一遍，然后进入等待状态；
- 一般把所有的输入信号都作为敏感信号。

always语句的语法格式

always语句的敏感信号列表有多种书写形式:

always @(A or B or C)

always @(A , B , C)

always @(*)

always @(posedge CLK)

always @(posedge CLK or negedge RST)

always语句的内部语句

- always内部的语句具有顺序执行的特点，也叫做顺序语句；
- 一条always语句内部可以包含一条顺序语句，也可以包含多条顺序语句；
- 多条顺序语句用begin...end括起来，一条顺序语句可以省略掉begin...end；
- always内部被赋值的变量必须是reg类型；
- always内部不能再出现过程语句；
- always内部的赋值符号有“=”“<=”两种。

【例 5-7】4 选 1 选择器

```
module MUX4to1(A, B, C, D, S, Y);
  input A, B, C, D;
  input [1:0] S;
  output  Y;
  reg Y;
  always @(A or B or C or D or S)
      begin
        case (S)
          2'b00: Y <= A;
          2'b01: Y <= B;
          2'b10: Y <= C;
          2'b11: Y <= D;
          Default: Y <= A;
        endcase
      end
endmodule
```

仿真中会遇到不带@的always语句，如：

```
always #10  CLK = ~ CLK;
```

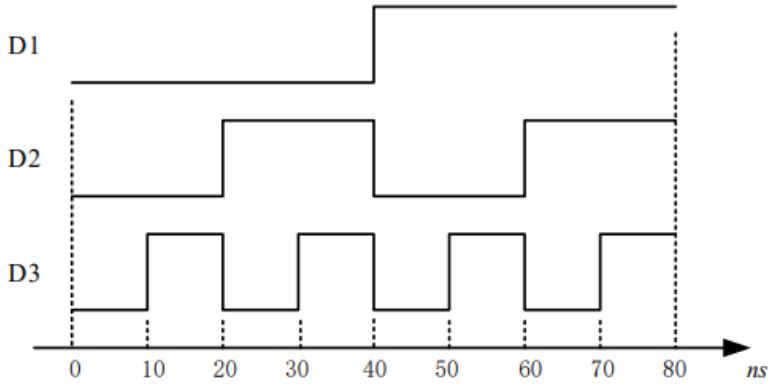
4.5.2.2 initial语句的语法格式

```
initial
    begin
        语句或语句块;
    end
```

initial语句的执行过程是：

- 从仿真的0时刻开始，顺序执行所有语句，而且只执行一遍；

【例 5-10】initial 语句的应用实例




```
`timescale 1ns/100ps //仿真时间标度语句
module test;
  reg D1, D2, D3;
  initial
      begin
          D1 = 0; D2 = 0; D3 = 0; //从 0 时刻执行，三个变量都赋值 0
          # 10  D3 = 1; //过了 10ns，D3 赋值为 1，其他两个不变
          # 10  D2 = 1; D3 = 0; //又过了 10ns，D2 赋值为 1，D3 赋值为 0，D1 不变
          # 10  D3 = 1; //又过了 10ns，D3 赋值为 1，其他两个不变
          # 10  D1 = 1; D2 = 0; D3 = 0;
          # 10  D3 = 1;
          # 10  D2 = 1; D3 = 0;
          # 10  D3 = 1;
          # 10  $finish //$finish 是系统函数，退出仿真
      end
endmodule
```

4.5.2.3 其他行为描述语句

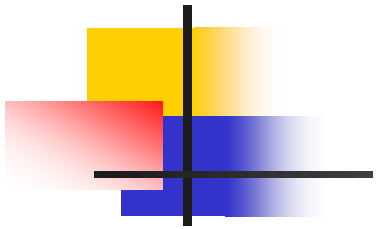
always语句和initial语句是复合结构，其内部还可以包含其他的行为描述语句，包括：

- 过程赋值语句
- if条件语句
- case条件语句
- 循环语句（for语句、while语句、repeat语句、forever语句）

1.过程赋值语句

- 过程赋值语句有两类：阻塞式赋值语句和非阻塞式赋值语句；
- 阻塞式赋值语句采用“=”符号；一条语句在执行时，其他赋值语句都被“阻塞”，即不能执行；
- 一条阻塞式语句执行完以后，目标变量即得到新的赋值，例如：

```
T1 = A & B; //此句执行时，其他语句不能执行。该句执行完，
            //T1获得新值。
T2 = B & Cin; //上一句执行完，再执行此句。
```



- 非阻塞式赋值语句采用“<=”符号，它在执行时，不阻塞其他语句的执行，因此多条语句可并行执行；
- 多条语句都要等到过程结束时才能完成赋值。例如：

```
T1 <= A & B; //此句执行时，其他语句也可以执行。
                    //过程结束时，T1获得新值。
T2 <= B & Cin; //可以与上一句同时执行，
                    //过程结束时，T2获得新值。
```

- 可以把多条非阻塞式赋值语句的执行看成是顺序“启动”，到过程语句结束时一起完成。

两种过程赋值语句的区别

- 阻塞式赋值语句和非阻塞式赋值语句在某些情况下可以互换，但一般情况下，它们产生不同的结果。

【例 5-12】非阻塞式赋值语句实例

```
module EXAMPLE_NONBLOCKING(A, B, C, D);
  input A, B;
  output  C, D;
  reg C, D;
  always @(A, B)
  begin
      C <= A & B;
      D <= A ^ B;
  end
endmodule
```




```
C = A & B;
D = A ^ B;
```

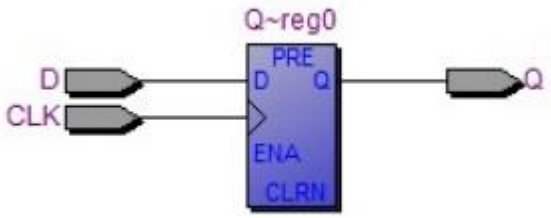
两种过程赋值语句的区别

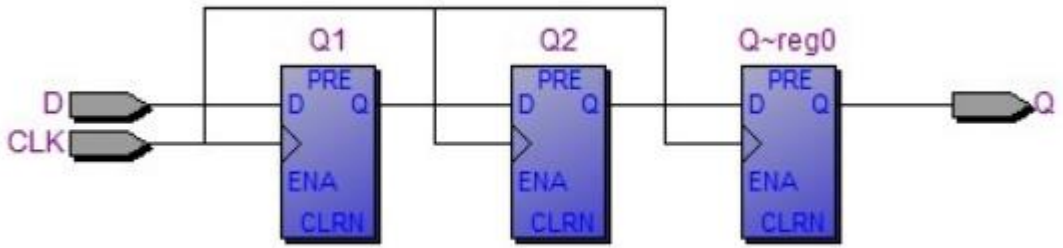
【例 5-13】阻塞式赋值语句描述 D 触发器

```
module EXAMPLE_B(CLK, D, Q);
  input CLK, D;
  output  Q;
  reg Q;
  reg Q1,Q2;
  always @(posedge CLK)
  begin
      Q1 = D;
      Q2 = Q1;
      Q = Q2;
  end
endmodule
```

【例 5-14】非阻塞式赋值语句描述 D 触发器

```
module EXAMPLE_NB(CLK, D, Q);
  input CLK, D;
  output  Q;
  reg Q;
  reg Q1,Q2;
  always @(posedge CLK)
  begin
      Q1 <= D;
      Q2 <= Q1;
      Q <= Q2;
  end
endmodule
```





2. if条件语句

if条件语句只能放在过程语句中，一般语法格式有三种：

格式一

```
if （条件表达式）
  begin
      语句块;
  end
```

格式二

```
if （条件表达式）
    begin
        语句块1;
    end
else
    begin
        语句块2;
    end
```

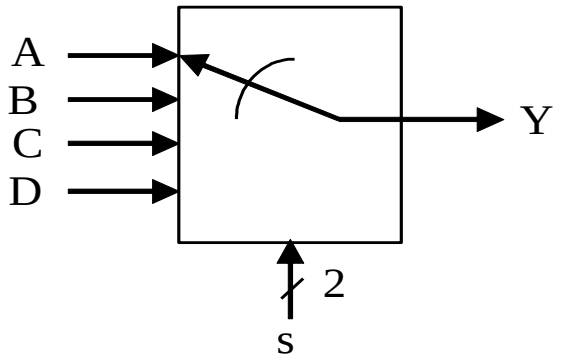
格式三

```
if （条件表达式1）
    begin
        语句块1;
    end
else if （条件表达式2）
    begin
        语句块2;
    end
……
else
    begin
        语句块n;
    end
```

- 条件表达式必须要加括号；
- 条件表达式1为“真”，则执行语句块1，否则，判断条件表达式2，为“真”，则执行语句块2；
- 如果所有列出的条件表达式都为“假”，则执行语句块n；
- if语句可以嵌套，但要注意else的归属，可用begin...end明确界定。

【例 5-17】if 语句实现四选一选择器

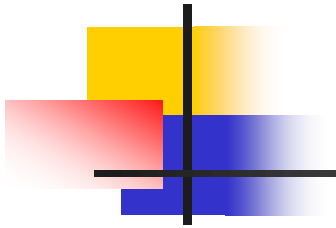
```
module SEL4(A, B, C, D, S, Y);
  input A, B, C, D;
  input [1:0] S;
  output  Y;
  reg Y;
    always @(A, B, C, D, S) begin
      if (S==2'b00)
          Y <= A;
      else if (S==2'b01)
          Y <= B;
      else if (S==2'b10)
          Y <= C;
      else
          Y <= D;
    end
endmodule
```



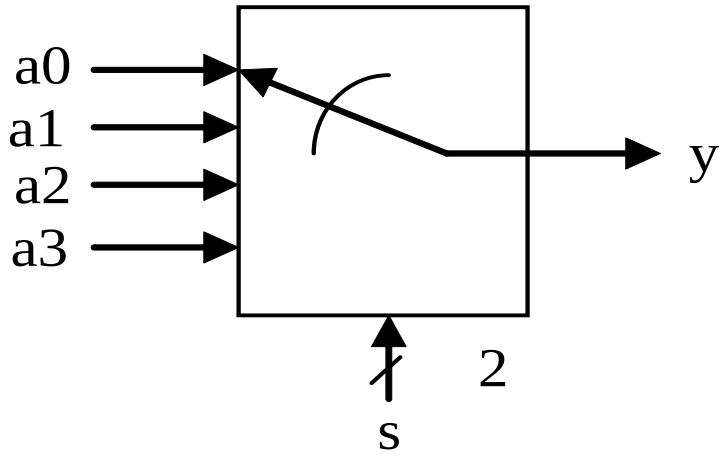
3. case条件语句

case条件语句只能放在过程语句中，一般语法格式为：

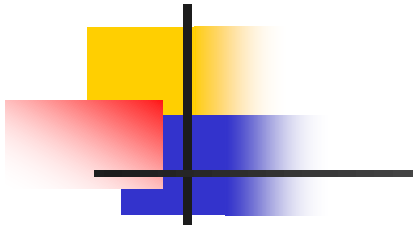
```
case （条件表达式）
    取值1：begin 语句块1；end
    取值2：begin 语句块2；end
      ……
    取值n：begin 语句块n；end
    [default：begin 语句块n+1；end]
endcase;
```



- case语句的执行过程：首先计算条件表达式的值，然后将它与后面各项分支取值进行对比，有相等的值就执行后面相应的语句块。执行完就跳出case语句；
- 如果case语句列出的各项取值可以覆盖条件表达式的取值范围，则可以省略掉default部分。否则，应加上default部分；
- 各项取值并不互相排斥，但只有第一个匹配的值对应的语句块被执行，执行完就跳出case语句；。

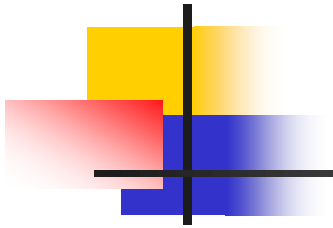



```
case  (s)
    2'b00 :   y = a0;
    2'b01 :   y = a1;
    2'b10 :   y = a2;
    2'b11 :   y = a3;
   default:     y = 1'bx;
endcase;
```



- case语句对条件表达式和各项分支取值逐位进行对比，把x、z也当作固定取值来对比；如果两者位宽不同，则对位宽小的值进行左边补0，然后对比；
- casez语句忽略掉值中的z，即不对z值做比较，直接忽略，其他同case语句。
- casex语句忽略掉值中的x和z，即不对x、z值做比较，其他同case语句。

```
casez  (mask)
    4'b1zzz :   Ans = 4'b1000;
    4'b01zz :   Ans = 4'b0100;
    4'b001z :   Ans = 4'b0010;
    4'b0001 :   Ans = 4'b0001;
    default:     Ans = 4'b0000;
endcase;
```



4. 循环语句

- 循环语句的作用是重复地执行一个语句块；
- 在硬件描述语言中，循环语句会增加系统资源的开销，同时增加设计难度；
- 一般用于算法实现，不用于控制；
- 循环语句分为四种：**for**语句、**while**语句、**repeat**语句和**forever**语句。

（1）for语句

for语句的语法格式为：

```
for （循环变量初始值表达式；循环控制条件表达式；循环变量修改表达式）
        循环体语句块；
```

循环变量一般是integer类型，在循环语句之前定义。

（1）for语句

for语句的执行步骤是：

a）由“循环变量初始值表达式”计算循环变量的初始值；

b）计算“循环控制条件表达式”的值，若为“真”，则执行循环体语句块，并转到步骤c；若为“假”，则结束循环语句；

c）按照“循环变量修改表达式”更改循环变量的值。可以采取递增或递减的修改方式，然后转到步骤b，继续执行。

【例 5-19】for 语句实现 4×4 乘法器

```
module MUL4_4_1(X, Y, R);
  input [3:0] X, Y;
  output [7:0] R;
  reg [7:0] R;
  integer i;
  always @(X or Y)
    begin
      R = 0;
      for (i=1; i<=4; i=i+1)
          if (Y[i-1])
              R = R + (X<<(i-1));
    end
endmodule
```

（2）while语句

while语句的语法格式为：

```
while （循环控制条件表达式）
        循环体语句块;
```

while语句的执行过程是：

- 计算“循环控制条件表达式”的值，若为“真”，就执行一遍循环体语句块；重复这个操作直到“循环控制条件表达式”的值为“假”，结束循环语句。
- 在while语句的循环体语句中，一定要有类似“循环变量修改”的表达式。

【例 5-20】while 语句实现 4×4 乘法器

```
module MUL4_4_2(X, Y, R);
  input [3:0] X, Y;
  output [7:0] R;
  reg [7:0] R;
  integer i;
  always @(X or Y)
    begin
      R = 0;
      i = 0;
      while (i<4) begin
        i = i + 1;
        if (Y[i-1])
          R = R + (X<<(i-1));
      end
    end
endmodule
```

（3）repeat语句

repeat语句的语法格式为：

```
repeat （循环次数表达式）
        循环体语句块;
```

repeat语句的执行过程是：

- 按照“循环次数表达式”给定的次数，重复执行循环体语句块。
- “循环次数表达式”可以是常数，也可以是变量，但在语句之前，变量要有确定的值。

【例 5-21】repeat 语句实现 4×4 乘法器

```
module MUL4_4_3(X, Y, R);
  input [3:0] X, Y;
  output [7:0] R;
  reg [7:0] R, XT;
  reg [3:0] YT;
  always @(X or Y)
    begin
      R = 0;
      XT = X;
      YT = Y;
      repeat (4) begin
        if (YT[0]) begin
          R = R + XT; end
        YT = YT >> 1;
        XT = XT << 1;
      end
    end
endmodule
```

（4）forever语句

forever语句的语法格式为：

forever　循环体语句块；

- forever语句表示连续不断地执行循环体语句块；
- 该语句不能综合，只能用在仿真中，产生周期性的激励信号。例如：

```
reg CLK;
……
initial begin
  CLK = 0;
  forever #10 CLK = ~ CLK;
end
```

4.5.3 任务和函数

- 在Verilog中，可以把一些重复性比较大的结构写成一个类似“子程序”的结构；
- 这样的结构有两类，一类叫做任务（task），另一类叫做函数（function）；
- Verilog程序可以在不同的位置调用同一个任务或函数。

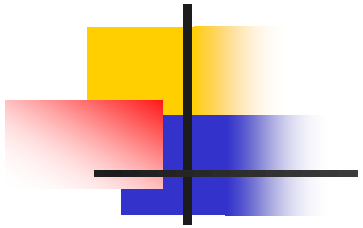
4.5.3.1 任务

任务定义的语法格式：

```
task  任务名；
    [端口及数据类型说明；]
    begin  语句块；end
endtask
```

任务调用的语法格式：

```
任务名 （端口1，端口2，…，端口n）；
```



- 任务定义在“task……endtask”之内；
- “任务名”是该任务在后续使用中的识别标志，任务名后面有分号；
- “端口及数据类型说明”是此任务的端口定义语句和数据类型说明；
- 任务中可以使用定义该任务的模块中的全局变量；
- 任务中的语句为顺序语句，不能出现always和initial这样的并行语句；
- 任务调用语句中的端口名的排列顺序要与任务定义中的端口顺序相匹配。

【例 5-22】任务定义和调用实例

```
module TASKDEMO(Din1, Din2, R);
  input [7:0] Din1, Din2;
  output [2:0] R;
  reg [2:0] R, Dout1, Dout2;
  task Bit_Num;
    input [7:0] X;
    output [2:0] Y;
    integer i;
    begin
      Y = 3'b000;
      for (i=0; i<=7; i=i+1)
        if (X[i])
          Y = Y + 1;
    end
  endtask

  always @(Din1, Din2)
    begin
      Bit_Num (Din1, Dout1);
      Bit_Num (Din2, Dout2);
    end

  assign R = (Dout1>=Dout2) ? Dout1 : Dout2;
endmodule
```

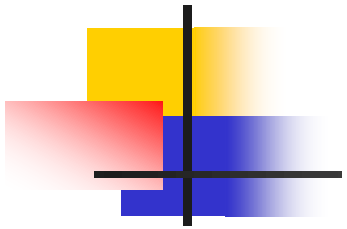
4.5.3.2 函数

函数定义的语法格式：

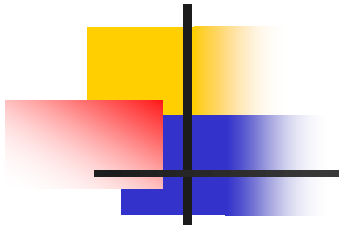
```
function [返回值位宽] 函数名；
    输入端口说明
    [其他说明；]
    begin 语句块；end
endfunction
```

函数调用时作为操作数，其语法格式为：

```
函数名 （端口1，端口2，…）
```



- 函数定义在“function……endfunction”之内；
- “返回值位宽”是返回值的类型和二进制位数。如“返回值位宽”，则默认为1位二进制；
- “输入端口说明”是此函数的输入端口的定义和数据类型说明；
- 函数可包含多个输入端口，但至少包含一个输入端口；
- 函数中的语句为顺序语句，不能出现always和initial这样的并行语句；



- 函数没有直接定义输出端口，但函数名就是一个隐含的寄存器类型输出变量；
- 函数调用是把函数名作为一个操作数来使用，没有独立的调用语句；
- **assign**语句和**always**语句、**initial**语句都可以调用函数；
- 一个函数中可以调用其他的函数，但不能调用任务。

【例 5-23】函数定义和调用实例

```
module FUNCTIONDEMO(Din1, Din2, R);
  parameter  N = 8;
  input [N-1:0] Din1, Din2;
  output [N-1:0] R;
  function [N-1:0] MAX_Num;
    input [N-1:0] X, Y;
    begin
      if (X>=Y)
          MAX_Num = X;
      else
          MAX_Num = Y;
    end
  endfunction
  assign R = MAX Num(Din1, Din2);
endmodule
```